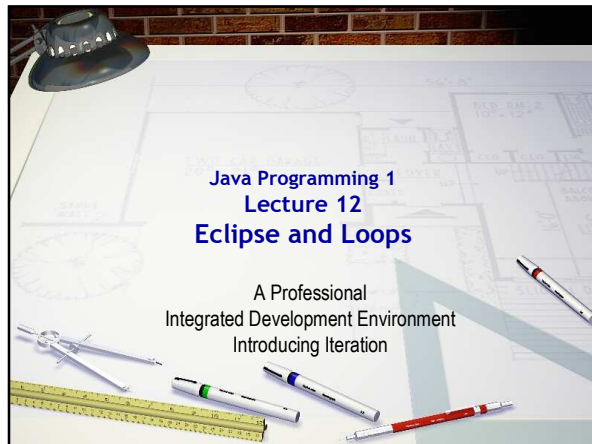
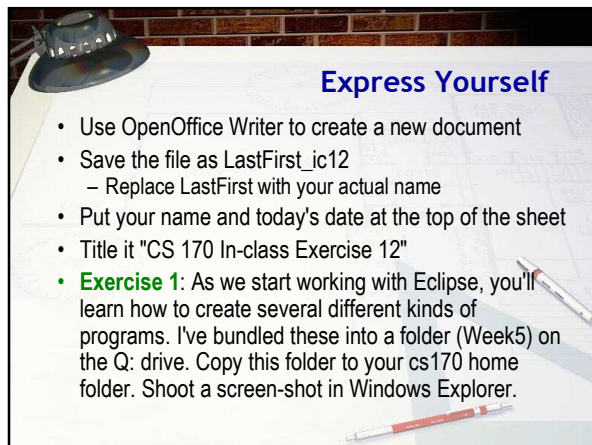


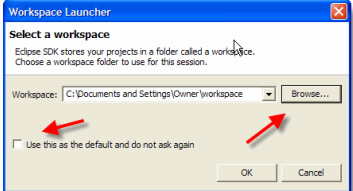






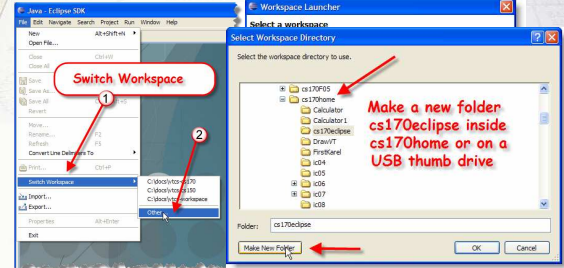
The Workspace

- The Eclipse **workspace** contains your files, projects, IDE customizations and preferences
 - Default lab workspace configuration is U:\eclipse
 - At home, though, you'll see this dialog at first



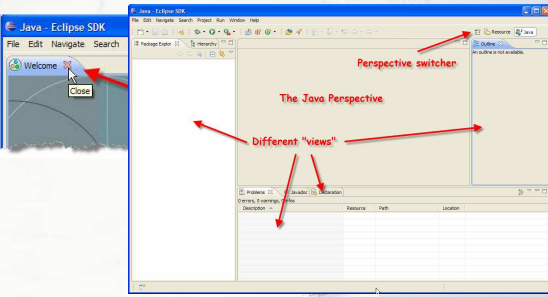
Switch Workspaces

- Switch workspaces with **File->Switch Workspace**
 - Can put on a "thumb drive" and carry around with you



The Java Perspective

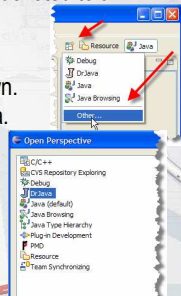
- Close the Welcome screen (first item on Help menu)



Package	Name	Resource	Path	Location

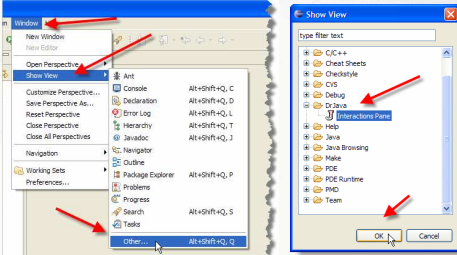
Perspectives

- Combination of windows and layout devoted to a specific task is called a **perspective**
- Exercise 3:** click Open Perspective button, choose Other from drop-down. Choose Resource and then Dr. Java. Shoot a screen-shot of each. Switch back to Java perspective.



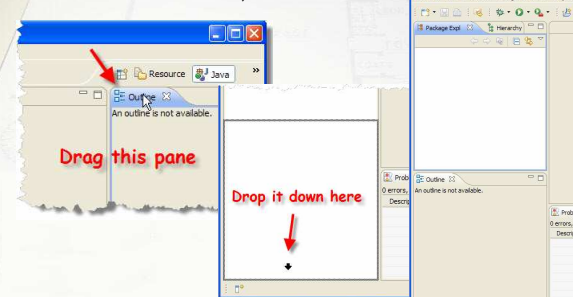
Views

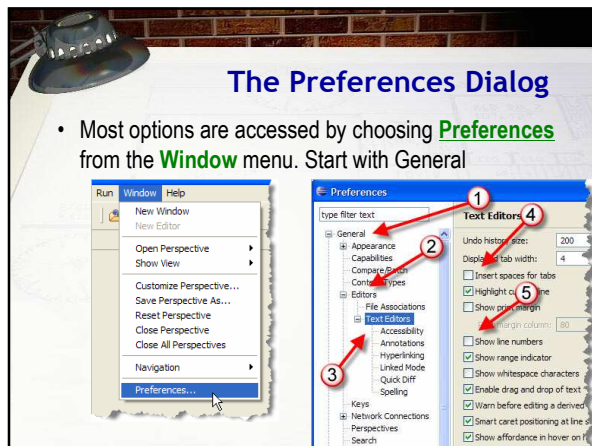
- Individual windows in a perspective are called **views**. You can add or remove a view from any perspective.

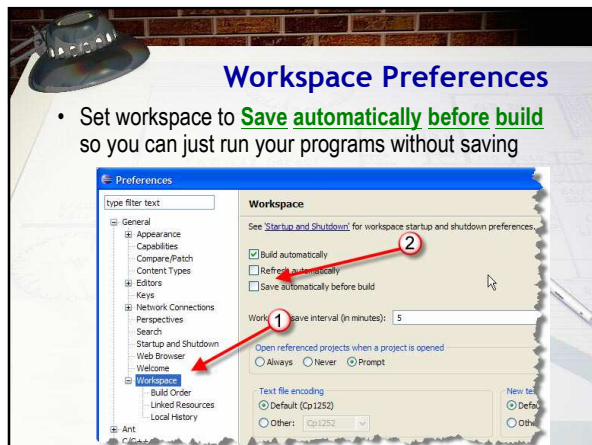


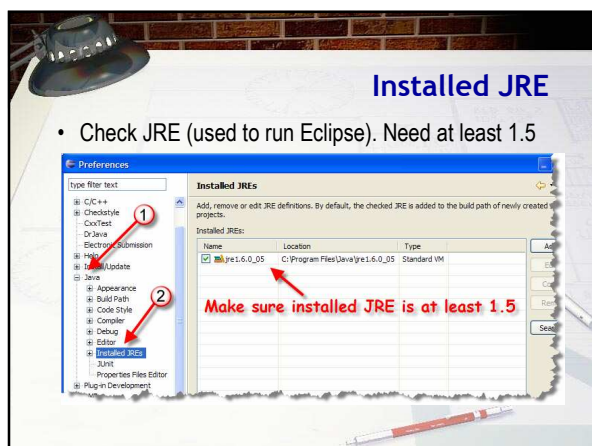
Arranging Views

- You can also move, float and dock individual views.



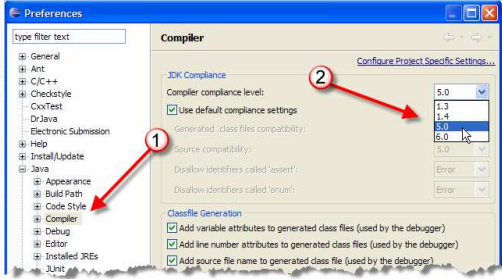






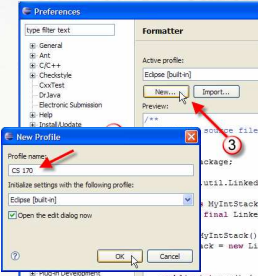
Compiler Conformance

- Used for generated code. Set to 5.0 for compatibility.



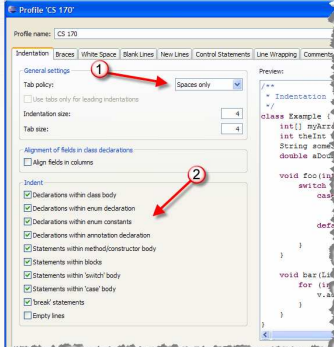
Code Formatting

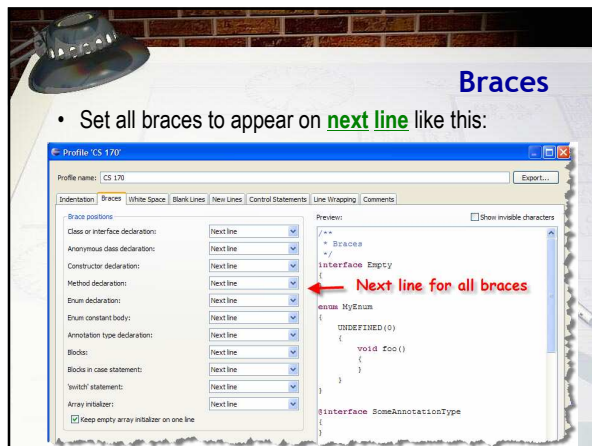
- Eclipse will automatically **format** your code for you
 - Built-in styles are different than class or text
- Create a formatting profile for CS 170
 - Choose Formatter under Code Style
 - Click New
 - Name the profile CS 170

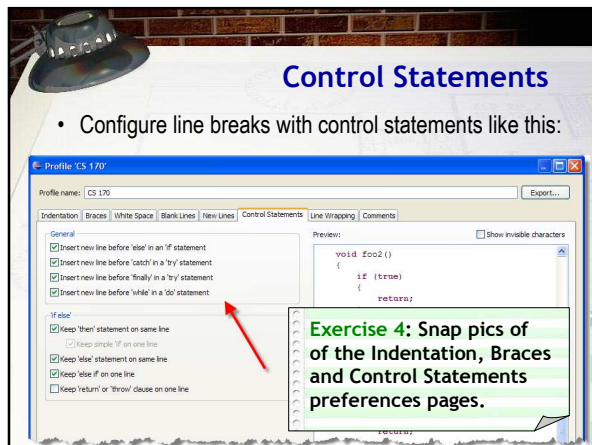


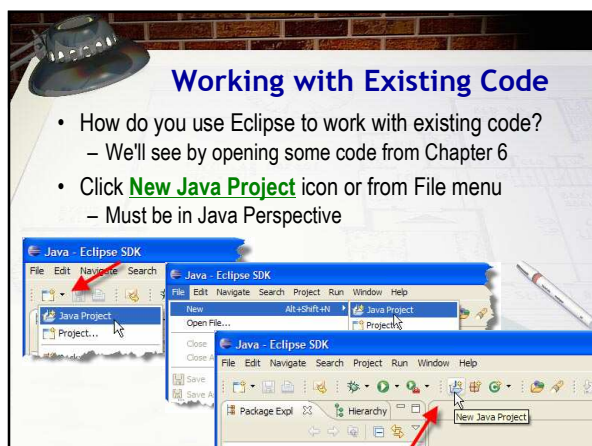
Indentation

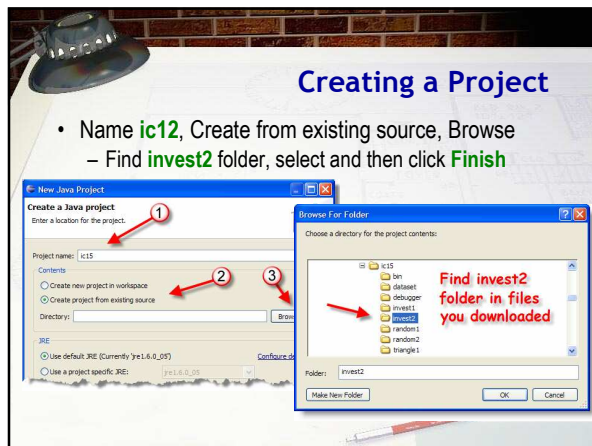
- Indentation
 - Spaces only
 - Select all checkboxes

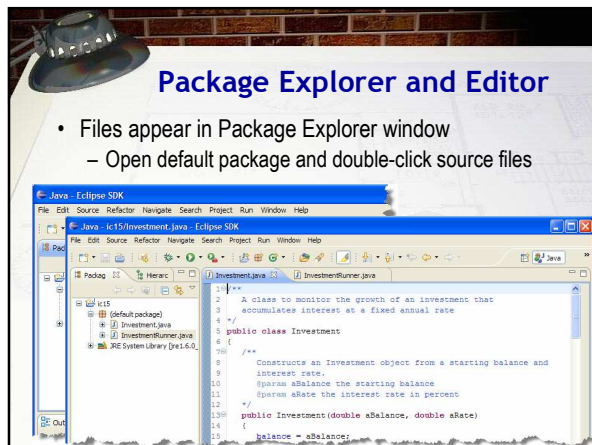


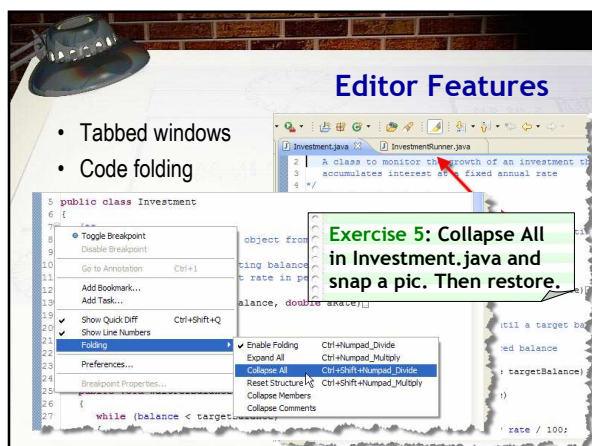


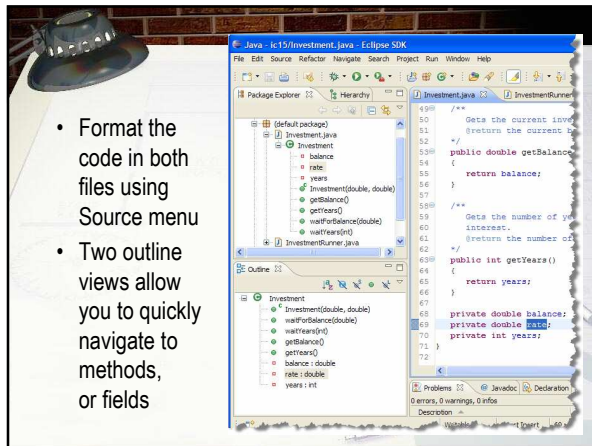




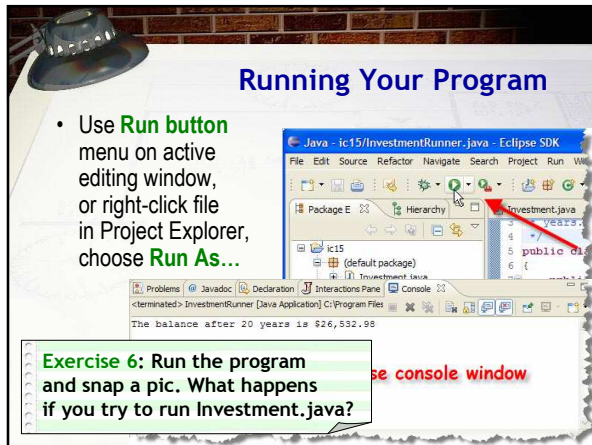








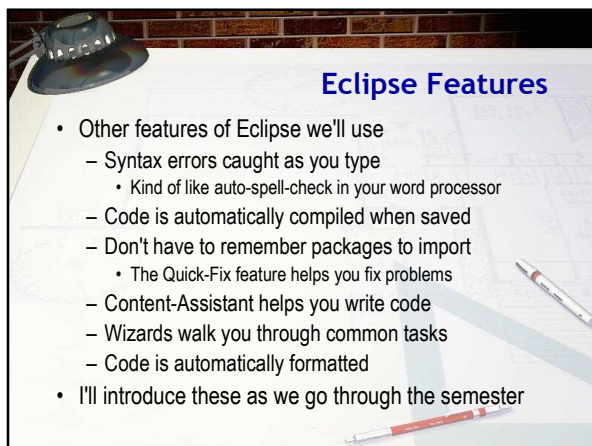
- Format the code in both files using Source menu
- Two outline views allow you to quickly navigate to methods, or fields



Running Your Program

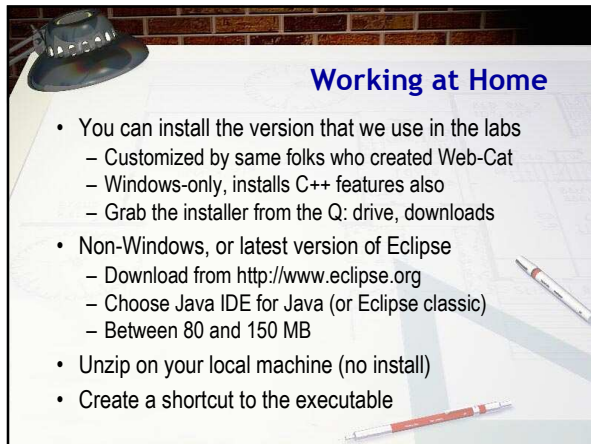
- Use **Run** button menu on active editing window, or right-click file in Project Explorer, choose **Run As...**

Exercise 6: Run the program and snap a pic. What happens if you try to run Investment.java?



Eclipse Features

- Other features of Eclipse we'll use
 - Syntax errors caught as you type
 - Kind of like auto-spell-check in your word processor
 - Code is automatically compiled when saved
 - Don't have to remember packages to import
 - The Quick-Fix feature helps you fix problems
 - Content-Assistant helps you write code
 - Wizards walk you through common tasks
 - Code is automatically formatted
- I'll introduce these as we go through the semester



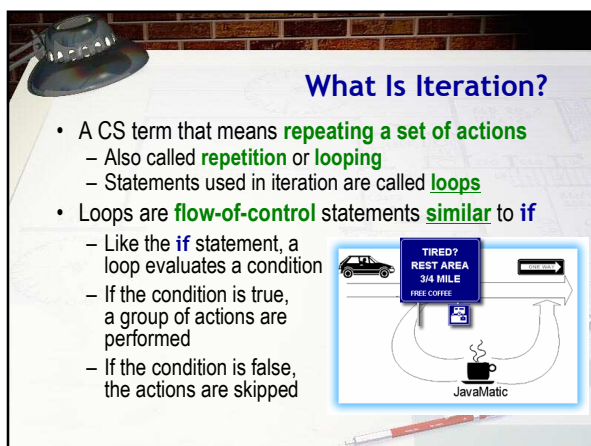
Working at Home

- You can install the version that we use in the labs
 - Customized by same folks who created Web-Cat
 - Windows-only, installs C++ features also
 - Grab the installer from the Q: drive, downloads
- Non-Windows, or latest version of Eclipse
 - Download from <http://www.eclipse.org>
 - Choose Java IDE for Java (or Eclipse classic)
 - Between 80 and 150 MB
- Unzip on your local machine (no install)
- Create a shortcut to the executable



Introducing Loops

LATHER
RINSE
REPEAT



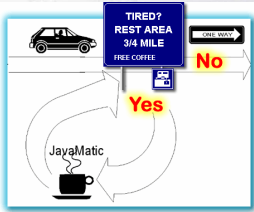
What Is Iteration?

- A CS term that means **repeating a set of actions**
 - Also called **repetition** or **looping**
 - Statements used in iteration are called **loops**
- Loops are **flow-of-control** statements **similar to if**
 - Like the **if** statement, a loop evaluates a condition
 - If the condition is true, a group of actions are performed
 - If the condition is false, the actions are skipped



Iteration and Selection

- Iteration is **not the same** as selection, however
- A loop **returns to its test** after performing its actions
- If the test is still true, then the actions are **performed again**
- A selection statement only performs its actions **once**
- You cannot replace a loop with a selection statement



Java and Iteration

- Java has three loops: **while**, **do-while**, and **for**
 - Each designed to work best in a particular situation
- One difference is **where** the test is made
- Test at the bottom**

```

{
    // Loop Body
}
Loop Test
        
```
- Test at the top**

```

Loop Test
{
    // Loop Body
}
        
```

Loop Classification

- Where tested is **not the best** method of classification
 - Doesn't provide any empirical guidance
 - Better to ask **how each loop is controlled**
- Two basic methods of loop control
 - Simplest method relies on a counter
 - Called **counting** or **counted** loops
 - "Repeat these actions 10 times"
 - More complex loops test the occurrence of a particular event, or combination of events.
 - These are called **indefinite** or **indeterminate** loops



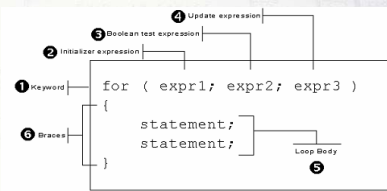
Counted Loops

- A counted loop is simplest kind of loop
 - The loop bounds is controlled by a counter
 - Use it to for "perform this action 10 times"
- Java's **for** loop **specializes** in this

```
for (int count = 0; count < 10; count++)  
{  
    System.out.println("count="+count);  
}
```

The for Loop Syntax

- The **for** loop has both a header and body (like **if**)

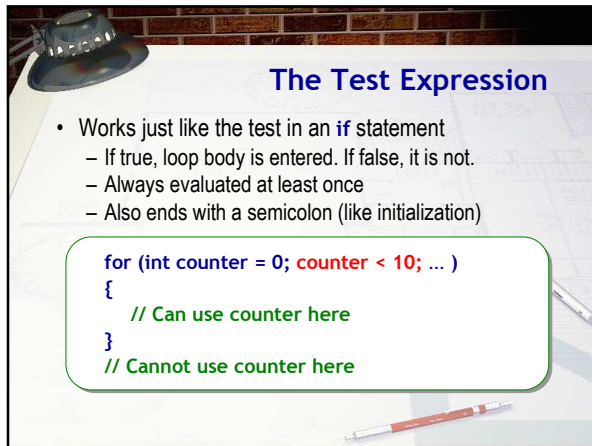


- Header has 3 expressions: **initialize**, **test**, and **update**

Initialization Expression

- The initialization expression
 - Executed once, before the loop is entered
 - Always executed, regardless of the test
 - Usually used to declare your counter variable

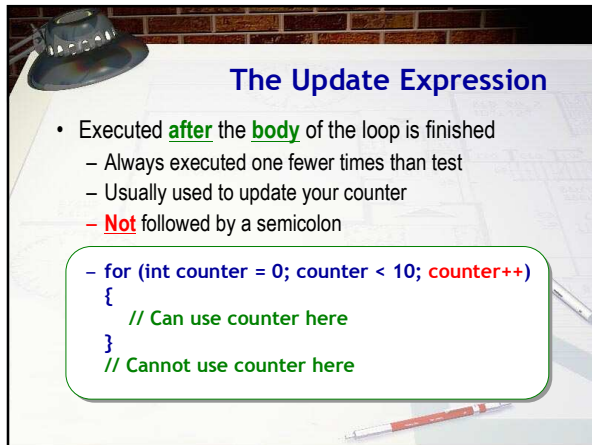
```
for (int counter = 0; ... ; ... )  
{  
    // Can use counter here  
}  
// Cannot use counter here
```



The Test Expression

- Works just like the test in an **if** statement
 - If true, loop body is entered. If false, it is not.
 - Always evaluated at least once
 - Also ends with a semicolon (like initialization)

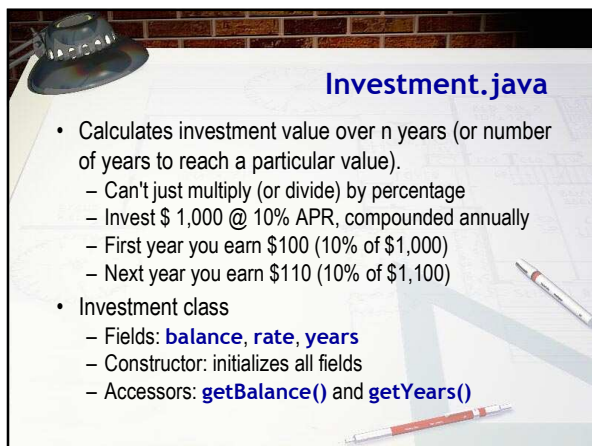
```
for (int counter = 0; counter < 10; ... )  
{  
    // Can use counter here  
}  
// Cannot use counter here
```



The Update Expression

- Executed **after** the **body** of the loop is finished
 - Always executed one fewer times than test
 - Usually used to update your counter
 - **Not** followed by a semicolon

```
- for (int counter = 0; counter < 10; counter++)  
{  
    // Can use counter here  
}  
// Cannot use counter here
```



Investment.java

- Calculates investment value over n years (or number of years to reach a particular value).
 - Can't just multiply (or divide) by percentage
 - Invest \$ 1,000 @ 10% APR, compounded annually
 - First year you earn \$100 (10% of \$1,000)
 - Next year you earn \$110 (10% of \$1,100)
- Investment class
 - Fields: **balance**, **rate**, **years**
 - Constructor: initializes all fields
 - Accessors: **getBalance()** and **getYears()**

The Future Value

- Using a **for** loop to calculate the value after n years

```

35 /**
36  * Keeps accumulating interest for a given number of years.
37  * @param n the number of years
38  */
39 public void waitYears(int n)
40 {
41     for (int i = 1; i <= n; i++)
42     {
43         double interest = balance * rate / 100;
44         balance = balance + interest;
45     }
46     years = years + n;
47 }
48
49 /**

```

InvestmentRunner

- Create an investment, wait n years, get the balance

```

5 public class InvestmentRunner
6 {
7     public static void main(String[] args)
8     {
9         final double INITIAL_BALANCE = 10000;
10        final double RATE = 5;
11        final int YEARS = 20;
12
13        Investment invest = new Investment(INITIAL_BALANCE, RATE);
14        invest.waitYears(YEARS);
15        double balance = invest.getBalance();
16
17        System.out.printf("The balance after %d years is %%,.2f\n", YEARS,
18                          balance);
19    }
20 }
21

```

Express Yourself

- Exercise 7:** some loop exercises at JavaBat.com

- Select the **doubleChar** exercise in the **String2** section

```

String2 > doubleChar
prev | next | chance
Given a string, return a string where for every char in the original, there are two chars.
doubleChar("The") -> "TThhee"
doubleChar("AAbb") -> "AAAbbBBb"
doubleChar("Hi-There") -> "HHi--TThheere"
Show Hint

```

- Questions you should ask?
 - Can we figure out how many times we need to loop?
 - If so, then we can use a counter-controlled for loop
 - What output is needed? What do we do inside the loop?

Loops and Strings

- Some tips to help you process **Strings** with loops
 - Use the **length()** method to visit every character
 - Use the **charAt()** or **substring()** method to extract
 - The **indexOf()** method is also useful

```
int len = str.length();
for (int i = 0; i < len; i++)
{
    char ch = str.charAt(i);
    String strCh = str.substring(i, i+1);
}
```

Another Exercise

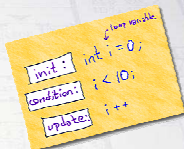
- Exercise 8:** another loop exercises at JavaBat.com
 - Select the **countHi** exercise in the **String2** section
- Use **substring(i, i+2)** and the **equals()** method
- Don't want to go through entire string
 - Can end up looking "past end" which causes a crash
- No need for special case



String2 > countHi
prev | next | chance
Return the number of times that the string "hi" appears anywhere in the given string.
countHi("abc hi ho") → 1
countHi("abc hi hi") → 2
countHi("hi hi") → 2
Show Hint

Introducing the while Loop

- Counted loops with **for** are easy
- All of the pieces are in one place
 - The initializer for your counter
 - The test expression
 - The update expression
- The simpler, general-purpose **while** loop can **also** be used to create counted loops
 - It's just not quite as convenient as **for**



init: int i = 0;
condition: i < 10;
update: i++

The while Loop Syntax

- Parts of a **while**
 - Keyword **while**, a boolean test, loop body

```
1 while ( 2 condition ) 3 { 4 statement; 4 statement; }
```

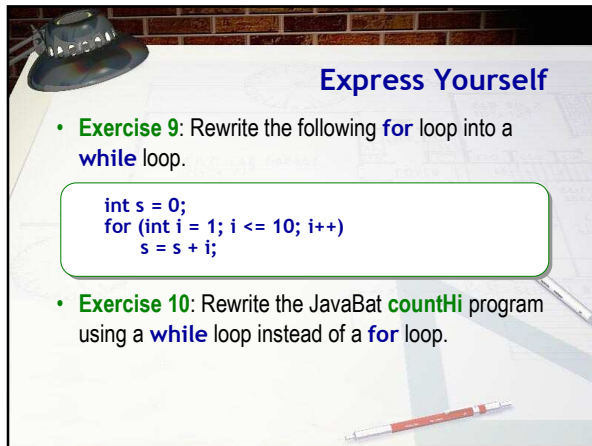
Counted while Loops

- To use the **while** statement to build a counted loop, you must do three things:
 - 1. **Create** a counter variable, and initialize it *before* the loop is encountered.
 - 2. **Test** the counter variable inside the **while** loop's boolean condition expression
 - 3. **Update** your counter at the **end** of the loop body. This takes the place of the **for** loop's update expression.

A Counted while Skeleton

- Here is a skeleton for building counted **while** loops:

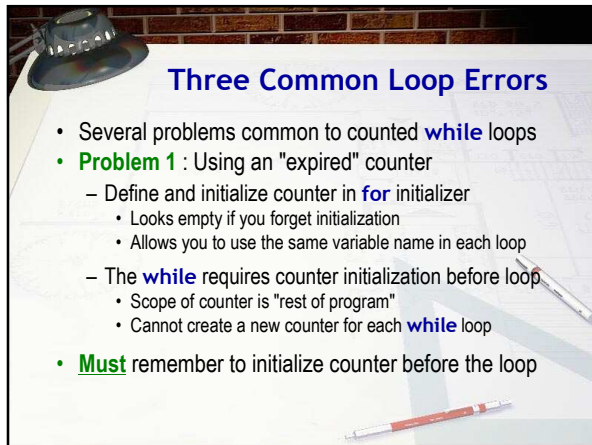
```
- int counter = 0;
while (counter < 10)
{
    // Loop body statements
    counter++;
}
```



Express Yourself

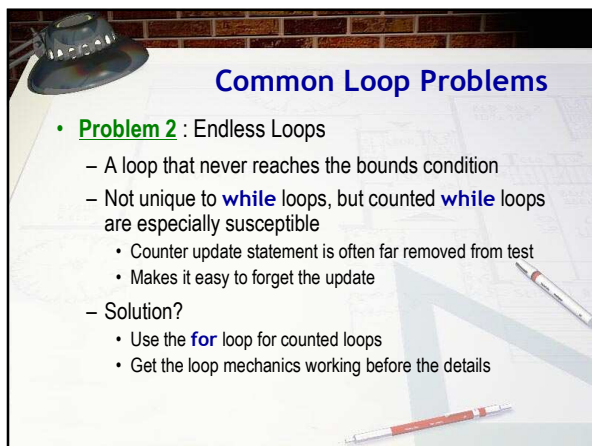
- **Exercise 9:** Rewrite the following **for** loop into a **while** loop.

```
int s = 0;
for (int i = 1; i <= 10; i++)
    s = s + i;
```
- **Exercise 10:** Rewrite the JavaBat **countHi** program using a **while** loop instead of a **for** loop.



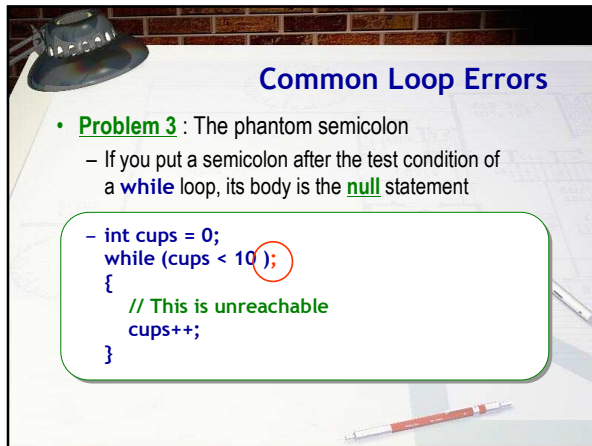
Three Common Loop Errors

- Several problems common to counted **while** loops
- **Problem 1:** Using an "expired" counter
 - Define and initialize counter in **for** initializer
 - Looks empty if you forget initialization
 - Allows you to use the same variable name in each loop
 - The **while** requires counter initialization before loop
 - Scope of counter is "rest of program"
 - Cannot create a new counter for each **while** loop
- **Must** remember to initialize counter before the loop



Common Loop Problems

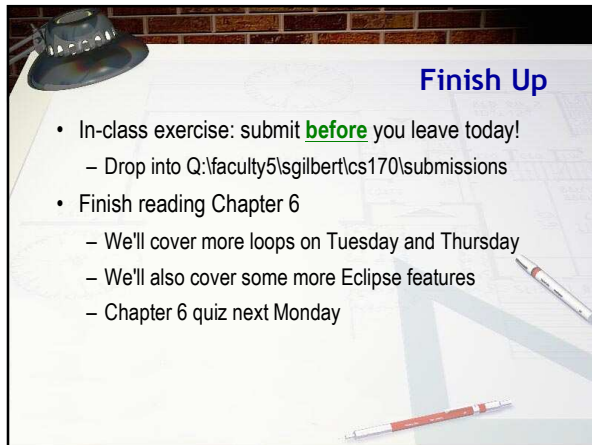
- **Problem 2:** Endless Loops
 - A loop that never reaches the bounds condition
 - Not unique to **while** loops, but counted **while** loops are especially susceptible
 - Counter update statement is often far removed from test
 - Makes it easy to forget the update
 - Solution?
 - Use the **for** loop for counted loops
 - Get the loop mechanics working before the details



Common Loop Errors

- **Problem 3** : The phantom semicolon
 - If you put a semicolon after the test condition of a **while** loop, its body is the **null** statement

```
int cups = 0;
while (cups < 10);
{
    // This is unreachable
    cups++;
}
```



Finish Up

- In-class exercise: submit **before** you leave today!
 - Drop into Q:\faculty5\sgilbert\cs170\submissions
- Finish reading Chapter 6
 - We'll cover more loops on Tuesday and Thursday
 - We'll also cover some more Eclipse features
 - Chapter 6 quiz next Monday
